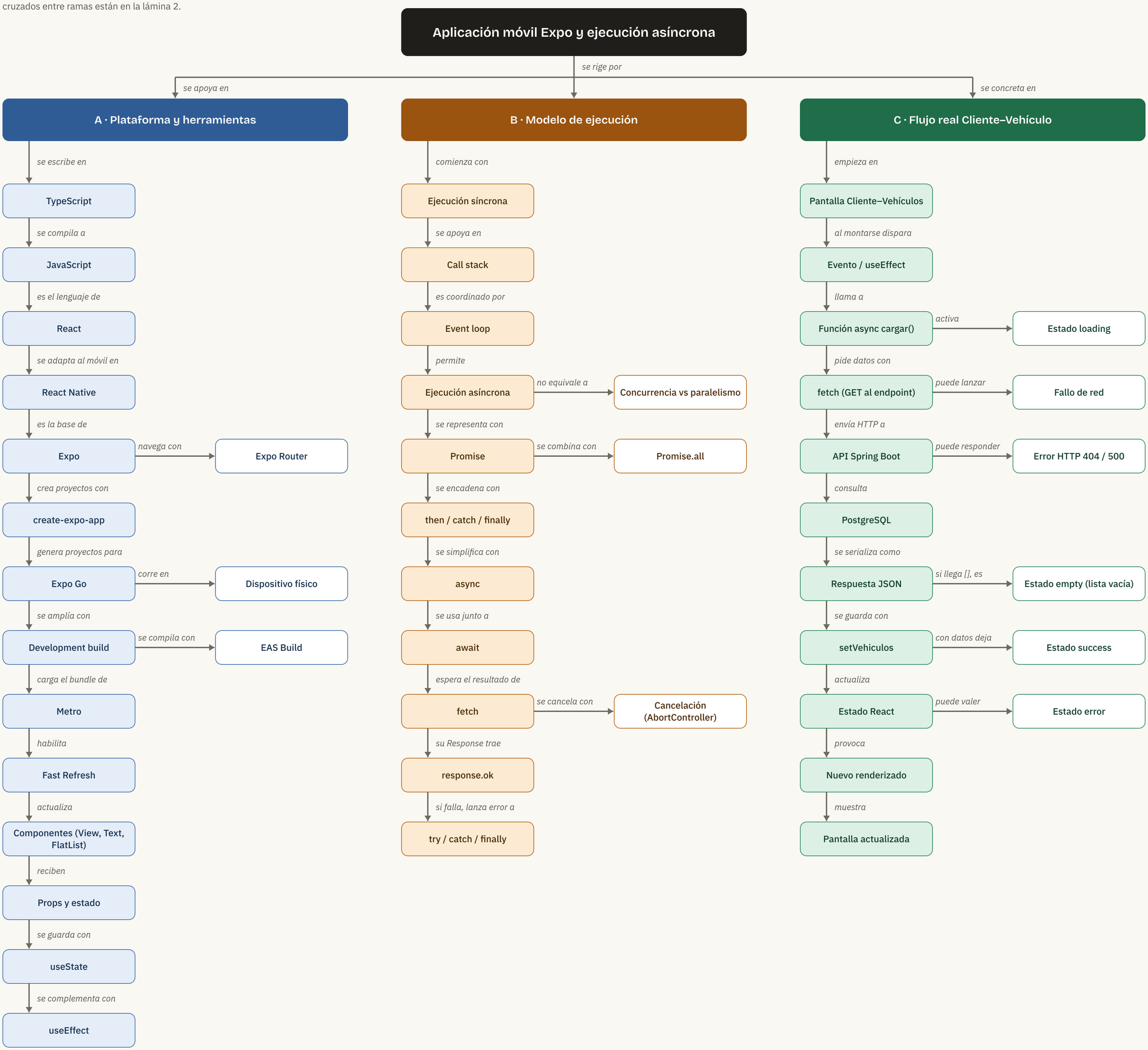
Pregunta focal: ¿Cómo Expo permite mostrar datos de una API sin bloquear la interfaz?

Cada flecha se lee de arriba hacia abajo: concepto, palabra de enlace, concepto. Los enlaces cruzados entre ramas están en la lámina 2.

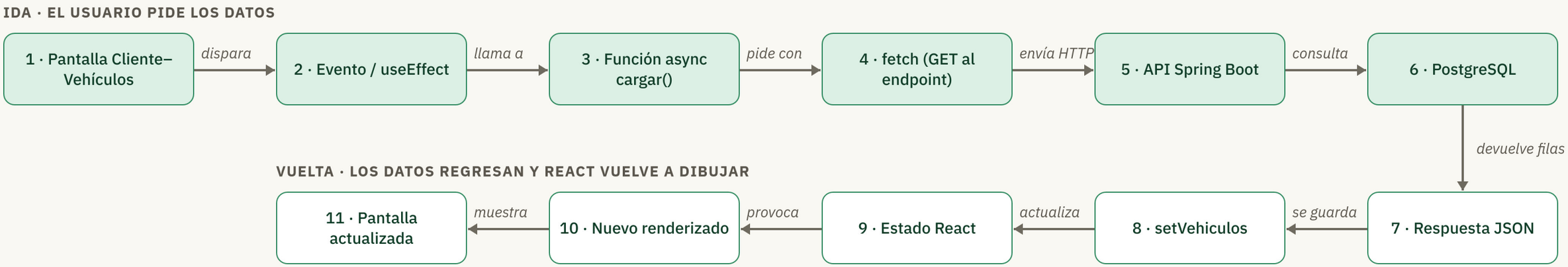


Enlaces cruzados entre ramas

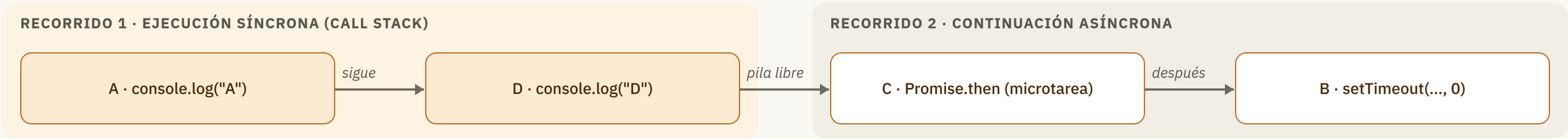
Doce proposiciones que conectan A (azul), B (naranja) y C (verde). Cada flecha se lee de izquierda a derecha como una oración verdadera.

1	useEffect	*puede iniciar*	fetch	A → B
2	fetch (GET al endpoint)	*devuelve una*	Promise	C → B
3	await	*suspende la continuación de*	Función async cargar()	B → C
4	setVehiculos	*actualiza*	Props y estado	C → A
5	Props y estado	*al cambiar, provoca*	Nuevo renderizado	A → C
6	JavaScript	*ejecuta sus funciones en el*	Call stack	A → B
7	Ejecución asíncrona	*evita congelar*	Pantalla Cliente–Vehículos	B → C
8	response.ok	*permite detectar*	Error HTTP 404 / 500	B → C
9	try / catch / finally	*en catch maneja*	Fallo de red	B → C
10	try / catch / finally	*en finally desactiva*	Estado loading	B → C
11	Dispositivo físico	*necesita red hacia*	API Spring Boot	A → C
12	useEffect	*puede cancelar con*	Cancelación (AbortController)	A → B

Recorrido de una consulta: pantalla → API → base de datos → pantalla



Dos recorridos temporales: orden de ejecución A, D, C, B



A y D se ejecutan de forma síncrona sobre el call stack. La continuación de la Promise se procesa antes que el callback del temporizador. setTimeout(..., 0) no significa «inmediato»: significa «cuando el call stack esté libre y el event loop llegue a esa tarea». Orden típico: A, D, C, B.

Estados de UI que la pantalla debe mostrar de forma explícita

loading
Hay una solicitud en curso: mostrar un indicador de carga.

success
Llegaron vehículos: mostrar la lista con FlatList.

empty
La respuesta fue exitosa pero la lista está vacía: mostrar «Sin vehículos».

error
response.ok falso o fallo de red: mostrar un mensaje y permitir reintentar.

setLoading(true) activa loading; el bloque finally lo desactiva siempre, haya éxito o error.

Tres ejemplos y un escenario de error

Ejemplo 1 · Orden de ejecución

```
console.log("A");
setTimeout(() => console.log("B"), 0);
Promise.resolve().then(() => console.log("C"));
console.log("D");
// salida típica: A, D, C, B
```

A y D son síncronos. La continuación de la Promise corre antes que el temporizador, y setTimeout con 0 no es inmediato.

Ejemplo 2 · async, await y response.ok

```
async function obtenerVehiculos(clienteId) {
  const r = await fetch(`${API_URL}/clientes/${clienteId}/
vehiculos`);
  if (!r.ok) throw new Error(`HTTP ${r.status}`);
  return await r.json();
}
```

La función async siempre devuelve una Promise. Un 404 o 500 no rechaza fetch, por eso se comprueba response.ok. La URL es ilustrativa: confirmar el endpoint real en Swagger.

Ejemplo 3 · try / catch / finally y estados

```
async function cargar(clienteId) {
  setLoading(true); setError(null);
  try { setVehiculos(await obtenerVehiculos(clienteId)); }
  catch (e) { setError("No se pudieron cargar vehículos"); }
  finally { setLoading(false); }
}
```

finally apaga loading tanto en éxito como en error. Una lista vacía no es un error: la pantalla muestra el estado empty.

Escenario de error · se cae la red

1 · fetch no recibe respuesta

Su Promise se rechaza porque no hay Response.

2 · await relanza el rechazo

Dentro de la función async se comporta como una excepción.

3 · entra catch

setError("No se pudieron cargar vehículos") guarda el mensaje en el estado.

4 · se ejecuta finally

setLoading(false) apaga el indicador de carga.

5 · React vuelve a renderizar

La pantalla muestra el estado error con opción de reintentar.

Si el servidor responde 404 o 500, fetch sí devuelve Response: response.ok es falso, se lanza un Error y el flujo sigue igual desde el paso 3.